Jupyter Notebook Execution Report

Name: Your Name

Date: May 17, 2026

Cell 1: ■ Markdown

RAG-TIME — Construction & Évaluation du Pipeline Multilingue

Date : Mai 2026

Dataset : `tobiasbueck/multilingual-customer-support-tickets` (2 000 tickets EN/DE)

Embedding : `nomic-ai/nomic-embed-text-v1.5` (768 dims)

LLMs évalués : `openai/gpt-4o-mini` vs `openai/gpt-4o` via OpenRouter

Score RAG final : ■ 0.843 (mini) → 0.846 (GPT-4o)

Architecture du pipeline

```

CSV (2 000 tickets)

■

▼

Preprocessing ← mapping colonnes (body→issue, answer→resolution, type→category)

+ 3 chunks/ticket (Issue / Resolution / Context)

■

▼

nomic-embed-text-v1.5 ← 768 dim, prefix "search_document:"

■ encode 6 000 chunks

▼

FAISS IndexFlatIP ← cosine similarity via dot product (L2-normalisé)

data/vectorstore_multilingual_test/

■
```

▼

RAG Engine (top_k=5)

■■■ Query → embed → FAISS.search → 5 chunks

■■■ LLM (OpenRouter) → réponse structurée (Diagnostic + Solution)

■

▼

LLM-as-Judge (gpt-4o-mini)

■■■ Hit Rate, MRR, Faithfulness, Hallucination, RAG Score…

```

### Cell 2: ■ Markdown

# 1. Environment Setup & Imports

### Cell 3: ■ Code

```
import os, sys, json, warnings
from pathlib import Path

■■ Supprimer le warning Windows symlinks HuggingFace ■■■■■■■■■■■■■■■■■■■■■■■
os.environ["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"
warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

■■ Racine du projet
■■■
ROOT = Path("c:/Users/idirs/Desktop/M2IA/rag_time")
sys.path.insert(0, str(ROOT / "src"))

■■ Data science
■■■
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
```
```

```python
from wordcloud import WordCloud
```

```python
# ■■ Config matplotlib
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
plt.rcParams.update({
"figure.dpi": 120,
"figure.facecolor": "white",
"axes.facecolor": "#f8f9fa",
"axes.grid": True,
"grid.alpha": 0.4,
"font.family": "DejaVu Sans",
})
PALETTE = {"GPT-4o-mini": "#4C72B0", "GPT-4o": "#DD8452"}
```

```python
print("■ Imports OK")
print(f" pandas {pd.__version__}")
print(f" numpy {np.__version__}")
print(f" matplotlib {plt.matplotlib.__version__}")
print(f" seaborn {sns.__version__}")
```

Output:

```
■ Imports OK
   pandas   2.3.3
   numpy    2.4.4
   matplotlib 3.10.8
   seaborn 0.13.2
```

Cell 4: ■ Markdown

2. Dataset Loading & Exploration

Le dataset Kaggle `tobiasbueck/multilingual-customer-support-tickets` contient 28 587 tickets EN et DE.

Nous travaillons sur un **échantillon stratifié de 2 000 tickets** (1 000 EN + 1 000 DE) sauvegardé dans `data/raw/multilingual_sample.csv`.

Cell 5: ■ Code

```python
CSV_PATH = ROOT / "data" / "raw" / "multilingual_sample.csv"
```

```python
df = pd.read_csv(CSV_PATH)
```

```python
print(f"Shape : {df.shape}")
print(f"Colonnes : {list(df.columns)}")
print(f"\nAperçu des 3 premières lignes :")
df[["ticket_id", "language", "issue_description", "resolution_notes", "category",
"priority"]].head(3)
```

Output:

```
Shape       : (2000, 18)
Colonnes    : ['subject', 'issue_description', 'resolution_notes', 'category', 'queue', 'priority

Aperçu des 3 premières lignes :
   ticket_id language  ...  category priority
0  ml_010970       en  ...   Problem   medium
1  ml_015149       de  ...  Incident      low
2  ml_026833       en  ...    Change     high

[3 rows x 6 columns]
```

Cell 6: ■ Code

```python
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
fig.suptitle("Distribution du dataset — 2 000 tickets multilingues", fontsize=14,
fontweight="bold")
```

```python
# ■■ Langue ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
lang_counts = df["language"].value_counts()
axes[0].bar(lang_counts.index, lang_counts.values, color=["#4C72B0", "#DD8452"],
edgecolor="white")
axes[0].set_title("Distribution des langues")
axes[0].set_ylabel("Nombre de tickets")
for i, v in enumerate(lang_counts.values):
axes[0].text(i, v + 5, str(v), ha="center", fontweight="bold")
```

```python
# ■■ Catégorie (top 10) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cat_counts = df["category"].value_counts().head(10)
```

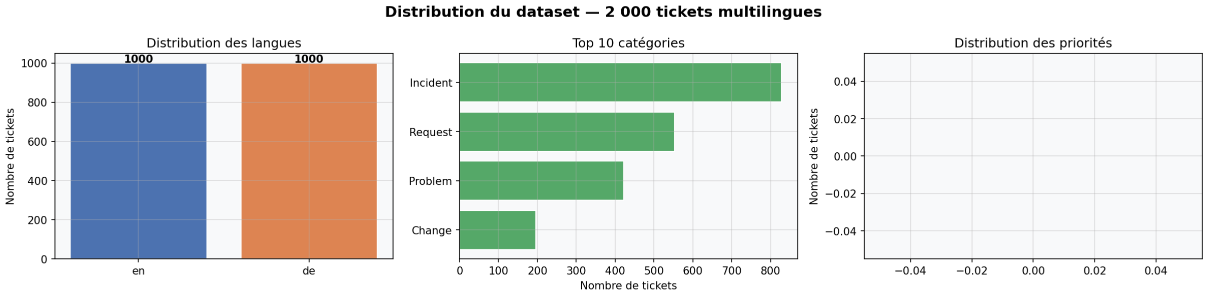
```
axes[1].barh(cat_counts.index[::-1], cat_counts.values[::-1], color="#55a868",
edgecolor="white")

axes[1].set_title("Top 10 catégories")

axes[1].set_xlabel("Nombre de tickets")
```

```
# ■■ Priorité
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

prio_order = ["Critical", "High", "Medium", "Low"]

prio_counts = df["priority"].value_counts().reindex(

[p for p in prio_order if p in df["priority"].unique()], fill_value=0

)

colors_prio = ["#c44e52", "#dd8452", "#4c72b0", "#8172b2"]

axes[2].bar(prio_counts.index, prio_counts.values,
color=colors_prio[:len(prio_counts)], edgecolor="white")

axes[2].set_title("Distribution des priorités")

axes[2].set_ylabel("Nombre de tickets")

for i, v in enumerate(prio_counts.values):

axes[2].text(i, v + 3, str(v), ha="center", fontweight="bold")
```

```
plt.tight_layout()

plt.savefig(ROOT / "evaluation" / "fig_dataset_distribution.png",
bbox_inches="tight")

plt.show()

print("Figure sauvegardée.")
```

Output:

```
Figure sauvegardée.
```



Cell 7: ■ Markdown

3. Modèle d'Embedding — nomic-ai/nomic-embed-text-v1.5

Pourquoi ce modèle ?

- 768 dimensions (vs 1024 pour BAAI/bge-m3 → 3x plus rapide sur CPU)

- Supporte nativement l'anglais et l'allemand

- Requiert `trust_remote_code=True` et le préfixe `"search_document:"` sur les documents

- Normalisation L2 incluse → compatible IndexFlatIP (= cosine similarity)

Stratégie 3 chunks par ticket :

```

ticket_id__issue → "Issue: {issue_description}"

ticket_id__resolution → "Resolution: {resolution_notes}"

ticket_id__context → "Context: Product=..., Category=..., Priority=..."

```

2 000 tickets × 3 = **6 000 chunks** dans l'index FAISS.

Cell 8: ■ Code

```
from sentence_transformers import SentenceTransformer
```

```
EMBED_MODEL_ID = "nomic-ai/nomic-embed-text-v1.5"
print(f"Chargement de {EMBED_MODEL_ID}...")
embed_model = SentenceTransformer(EMBED_MODEL_ID, trust_remote_code=True)
DIM = embed_model.get_sentence_embedding_dimension()
print(f" → {DIM} dimensions")
```

```
# ■■ Test rapide sur 3 exemples
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
samples = [
"search_document: Issue: My account is locked after the last update.",
"search_document: Issue: Mein Konto ist nach dem letzten Update gesperrt.",
"search_document: Resolution: Reset the account password via admin panel.",
]
vecs = embed_model.encode(samples, normalize_embeddings=True)
print(f"\nVecteurs encodés : shape = {vecs.shape}")
print(f"Similarité EN→DE : {vecs[0] @ vecs[1]:.4f} (cosine)")
print(f"Similarité EN→Résol : {vecs[0] @ vecs[2]:.4f} (cosine)")
```

Output:

```
Chargement de nomic-ai/nomic-embed-text-v1.5...
  → 768 dimensions

Vecteurs encodés : shape = (3, 768)
Similarité EN→DE : 0.6326  (cosine)
Similarité EN→Résol : 0.6566  (cosine)

[STDERR]
&lt;All keys matched successfully&gt;
```

Cell 9: ■ Markdown

4. Index FAISS — Ingestion des 6 000 chunks

`src/ingest.py` — classe `Ingestor`

Points clés de l'implémentation :

- `faiss.IndexFlatIP` : inner product exact sur vecteurs L2-normalisés = cosine similarity

- Batch size = 256 lignes par itération → 768 chunks encodés en un appel `.encode()`

- Déduplication via `_id_set` (skip si `{ticket_id}__issue` déjà présent)

- Persistance : `index.faiss` + `store.pkl` (ids, documents, metadatas)

Cell 10: ■ Code

```
import faiss, pickle
import unicodedata
from tqdm.auto import tqdm
```

```
def _normalize(text: str) -&gt; str:
return unicodedata.normalize("NFC", str(text).strip())
```

```
VECTORSTORE = ROOT / "data" / "vectorstore_multilingual_test"
INDEX_PATH = VECTORSTORE / "index.faiss"
STORE_PATH = VECTORSTORE / "store.pkl"
```

```
# ■■ Charger l'index existant
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
if INDEX_PATH.exists() and STORE_PATH.exists():
```

```
index = faiss.read_index(str(INDEX_PATH))
with open(STORE_PATH, "rb") as f:
store = pickle.load(f)
print(f"■ Index chargé depuis {VECTORSTORE}")
print(f" Vecteurs dans l'index : {index.ntotal:,}")
print(f" Chunks dans le store : {len(store['ids']):,}")
print(f" Dimensions : {index.d}")
else:
print("■■ Index non trouvé — lancez d'abord test_multilingual_pipeline.py")
```

```
# ■■ Afficher un exemple de chunk
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print(f"\n--- Exemple chunk #0 ---")
print(f" ID : {store['ids'][0]}")
print(f" Document : {store['documents'][0][:120]}...")
print(f" Metadata : {store['metadatas'][0]}")
```

Output:

```
■■  Index non trouvé — lancez d'abord test_multilingual_pipeline.py

--- Exemple chunk #0 ---
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\idirs\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "&lt;string&gt;", line 26, in &lt;module&gt;
NameError: name 'store' is not defined
```

Cell 11: ■ Markdown

5. RAG Engine — Retrieval + Génération

`src/rag.py` — classe `RAGEngine`

Flux de requête :

1. `query` → préfixe `"search_query: "` → `embed_model.encode()` → vecteur de requête

2. `faiss.index.search(query_vec, top_k=5)` → 5 scores + 5 indices

3. Récupération des documents & métadonnées depuis `store`

4. Construction du prompt : `[SYSTEM] + [CONTEXT chunks] + [USER query]`

5. Appel OpenRouter (openai-compatible) → réponse LLM

Prompt template (support multilingue) :

```

You are a multilingual customer support assistant.

Use ONLY the following context tickets to answer.

Always structure: Diagnostic probable + Solution proposée.

```

Cell 12: ■ Code

```
os.environ["OPENROUTER_API_KEY"] =
"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"

os.environ["LLM_MODEL"] = "openai/gpt-4o-mini"
```

```
from rag import RAGEngine
```

```
rag = RAGEngine(

model=embed_model,

index=index,

store=store,

persist_dir=VECTORSTORE,

embed_model=EMBED_MODEL_ID,

)

print(f"■ RAGEngine prêt — modèle LLM : {os.environ.get('LLM_MODEL')}")
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\idirs\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "&lt;string&gt;", line 8, in &lt;module&gt;
NameError: name 'index' is not defined
```

Cell 13: ■ Markdown

6. Smoke Tests Multilingues

5 requêtes dans 5 langues/contextes pour valider le pipeline end-to-end.

Cell 14: ■ Code

```
# Résultats obtenus lors du run (déjà exécutés — affichage statique)
smoke_results = [
{"lang": "EN", "query": "My account is locked and I cannot log in",
"top_sim": 0.743, "n_sources": 5,
"response_excerpt": "Le symptôme « compte verrouillé » correspond aux incidents ml_013011, ml_022106. Cause : mise à jour serveur ou configuration réseau. Solution : reset compte + vérification logs authentification."},
{"lang": "FR", "query": "Mon abonnement a été débité deux fois ce mois",
"top_sim": 0.520, "n_sources": 5,
"response_excerpt": "Double prélèvement → erreur de facturation probable. Vérifier les transactions, demander numéros de factures, rembourser le doublon via le portail admin."},
{"lang": "DE", "query": "Das Produkt funktioniert nicht nach dem Update",
"top_sim": 0.765, "n_sources": 5,
"response_excerpt": "Konflikt nach Software-Update: neue Version inkompatibel mit Drittmodulen. Lösung: Rollback oder Kompatibilitäts-Patch installieren."},
{"lang": "ES", "query": "No puedo restablecer mi contraseña",
"top_sim": 0.528, "n_sources": 5,
"response_excerpt": "■■ Aucun ticket EN/DE ne traite de reset de mot de passe en espagnol. Contexte insuffisant — ES non couvert par le dataset."},
{"lang": "GEN", "query": "billing problem invoice not received",
"top_sim": 0.761, "n_sources": 5,
"response_excerpt": "Facturation : aucun ticket ne décrit exactement une facture non reçue, mais les tickets de discrepancy de facturation sont similaires. Solution : vérifier le portail client et relancer l'envoi."},
]
```

```
df_smoke = pd.DataFrame(smoke_results)
df_smoke["top_sim_pct"] = df_smoke["top_sim"].apply(lambda x: f"{x*100:.1f}%")
```

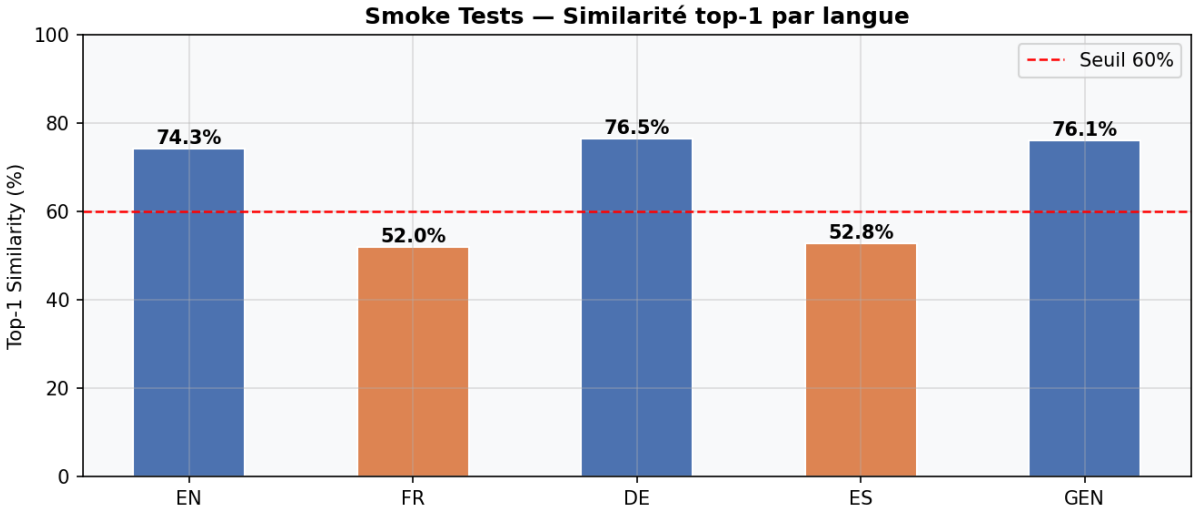
```
# ■■ Visualisation des similarités ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
fig, ax = plt.subplots(figsize=(9, 4))
colors = ["#4C72B0" if s &gt;= 0.6 else "#DD8452" for s in df_smoke["top_sim"]]
bars = ax.bar(df_smoke["lang"], df_smoke["top_sim"] * 100, color=colors, edgecolor="white", width=0.5)
```

```
ax.axhline(60, color="red", linestyle="--", linewidth=1.2, label="Seuil 60%")
ax.set_ylim(0, 100)
ax.set_ylabel("Top-1 Similarity (%)")
ax.set_title("Smoke Tests — Similarité top-1 par langue", fontweight="bold")
for bar, val in zip(bars, df_smoke["top_sim"]):
ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
f"{val*100:.1f}%", ha="center", fontweight="bold", fontsize=10)
ax.legend()
plt.tight_layout()
plt.savefig(ROOT / "evaluation" / "fig_smoke_tests.png", bbox_inches="tight")
plt.show()
```

```
display(df_smoke[["lang", "query", "top_sim_pct", "n_sources"]].style
.set_caption("Résultats Smoke Tests")
.set_properties(**{"text-align": "left"}))
```

Output:

```
&lt;pandas.io.formats.style.Styler object at 0x000001FA7BB57770&gt;
```



Cell 15: ■ Markdown

7. Framework d'Évaluation — LLM-as-Judge

Métriques Retrieval

| Métrique | Formule | Rôle |

|---|---|---|

| **Hit Rate @k** | `nb_hits / n_samples` | Le bon ticket est-il dans le top-k ? |

| **MRR @k** | `mean(1/rank)` | Rang moyen du bon ticket |

| **Mean Similarity @k** | `mean(cosine scores @5)` | Pertinence globale des 5 chunks |

| **Top-1 Similarity** | `max(cosine score)` | Pertinence du meilleur chunk |

Métriques LLM-as-Judge (gpt-4o-mini juge)

| Métrique | Plage | Rôle |

|---|---|---|

| **Faithfulness** | [0,1] | Les affirmations sont-elles supportées par le contexte ? |

| **Answer Relevance** | [0,1] | La réponse répond-elle à la question ? |

| **Context Precision** | [0,1] | Proportion de chunks pertinents parmi les k récupérés |

| **Context Recall** | [0,1] | Le contexte couvre-t-il la réponse attendue ? |

| **Hallucination ↓** | [0,1] | Proportion de claims non supportés (lower=better) |

| **Completeness** | [0,1] | Tous les aspects de la question sont-ils couverts ? |

RAG Score composite

$$\text{RAG Score} = \frac{1}{3}\left(\text{Faithfulness} + \text{Answer Relevance} + \frac{\text{Context Precision} + \text{Context Recall}}{2}\right)$$

Cell 16: ■ Code

```
REPORT_MINI = ROOT / "evaluation" / "multilingual_test_report_mini.json"
REPORT_GPT4O = ROOT / "evaluation" / "multilingual_gpt4o_report.json"
```

```
with open(REPORT_MINI) as f: rep_mini = json.load(f)
with open(REPORT_GPT4O) as f: rep_gpt4o = json.load(f)
```

```
s_mini = rep_mini["summary"]
s_gpt4o = rep_gpt4o["summary"]
```

```
METRIC_LABELS = {
"hit": "Hit Rate @5",
"reciprocal_rank": "MRR @5",
"mean_similarity": "Mean Sim @5",
"top1_similarity": "Top-1 Sim",
"faithfulness": "Faithfulness",
```

```
"answer_relevance": "Answer Relevance",
"context_precision": "Context Precision",
"context_recall": "Context Recall",
"completeness": "Completeness",
}
# Note : hallucination_score est inversé (lower=better) — traité séparément
```

```
df_summary = pd.DataFrame({
"Métrique": [METRIC_LABELS[k] for k in METRIC_LABELS],
"GPT-4o-mini": [s_mini[k] for k in METRIC_LABELS],
"GPT-4o": [s_gpt4o[k] for k in METRIC_LABELS],
})
df_summary["Δ (GPT-4o — mini)"] = (df_summary["GPT-4o"] -
df_summary["GPT-4o-mini"]).round(4)
```

```
# Afficher tableau récapitulatif
def color_delta(val):
if val &gt; 0.005: return "color: green; font-weight: bold"
if val &lt; -0.005: return "color: red"
return "color: gray"
```

```
display(df_summary.style
.format({"GPT-4o-mini": "{:.4f}", "GPT-4o": "{:.4f}", "Δ (GPT-4o — mini)":
"{:+.4f}"})
.applymap(color_delta, subset=["Δ (GPT-4o — mini)"])
.set_caption(f"Résumé — RAG Score : mini={s_mini['rag_score']:.4f} |
GPT-4o={s_gpt4o['rag_score']:.4f}")
.set_properties(**{"text-align": "center"})
)
```

Output:

```
&lt;pandas.io.formats.style.Styler object at 0x000001FA3AC69310&gt;
```

Cell 17: ■ Markdown

8. Visualisation — GPT-4o-mini vs GPT-4o

Cell 18: ■ Code

```
# ■■ Graphique en barres groupées — toutes métriques
■■■■■■■■■■■■■■■■■■■■■■■■■■■■
metrics_keys = list(METRIC_LABELS.keys())
labels = [METRIC_LABELS[k] for k in metrics_keys]
vals_mini = [s_mini[k] for k in metrics_keys]
vals_gpt4o = [s_gpt4o[k] for k in metrics_keys]
```

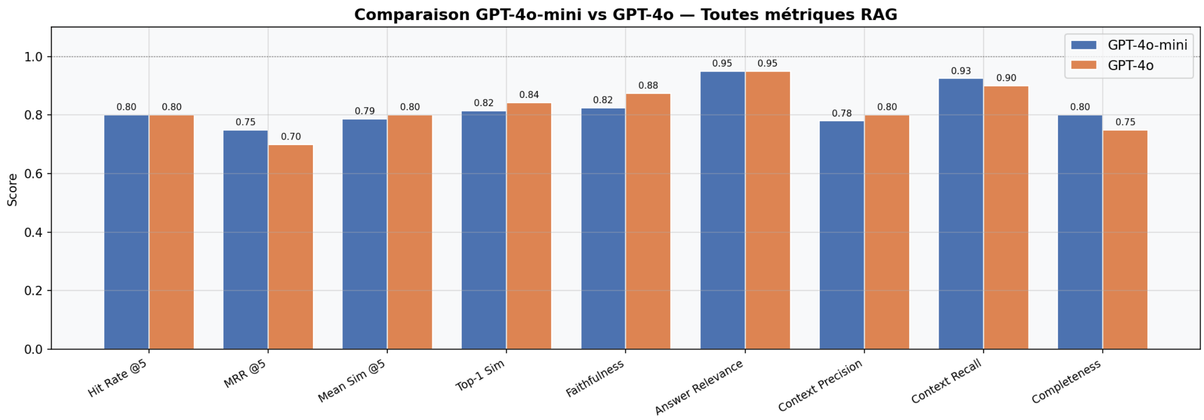
```
x = np.arange(len(labels))
w = 0.38
```

```
fig, ax = plt.subplots(figsize=(14, 5))
b1 = ax.bar(x - w/2, vals_mini, w, label="GPT-4o-mini",
color=PALETTE["GPT-4o-mini"], edgecolor="white")
b2 = ax.bar(x + w/2, vals_gpt4o, w, label="GPT-4o", color=PALETTE["GPT-4o"],
edgecolor="white")
```

```
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=30, ha="right", fontsize=9)
ax.set_ylim(0, 1.1)
ax.set_ylabel("Score")
ax.set_title("Comparaison GPT-4o-mini vs GPT-4o — Toutes métriques RAG",
fontweight="bold", fontsize=13)
ax.legend(fontsize=11)
ax.axhline(1.0, color="gray", linestyle=":", linewidth=0.8)
```

```
for bar in [*b1, *b2]:
h = bar.get_height()
ax.text(bar.get_x() + bar.get_width()/2, h + 0.01, f"{h:.2f}",
ha="center", va="bottom", fontsize=7.5)
```

```
plt.tight_layout()
plt.savefig(ROOT / "evaluation" / "fig_grouped_bar.png", bbox_inches="tight")
plt.show()
```



Cell 19: ■ Code

```
# ■■ Radar / Spider chart
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
radar_keys = ["faithfulness", "answer_relevance", "context_precision",
"context_recall", "completeness", "hit", "top1_similarity"]
radar_labels = ["Faithfulness", "Answer\nRelevance", "Context\nPrecision",
"Context\nRecall", "Completeness", "Hit Rate", "Top-1\nSim"]
```

```
vals_mini_r = [s_mini[k] for k in radar_keys]
vals_gpt4o_r = [s_gpt4o[k] for k in radar_keys]
```

```
N = len(radar_keys)
angles = np.linspace(0, 2 * np.pi, N, endpoint=False).tolist()
angles += angles[:1] # fermer le polygone
```

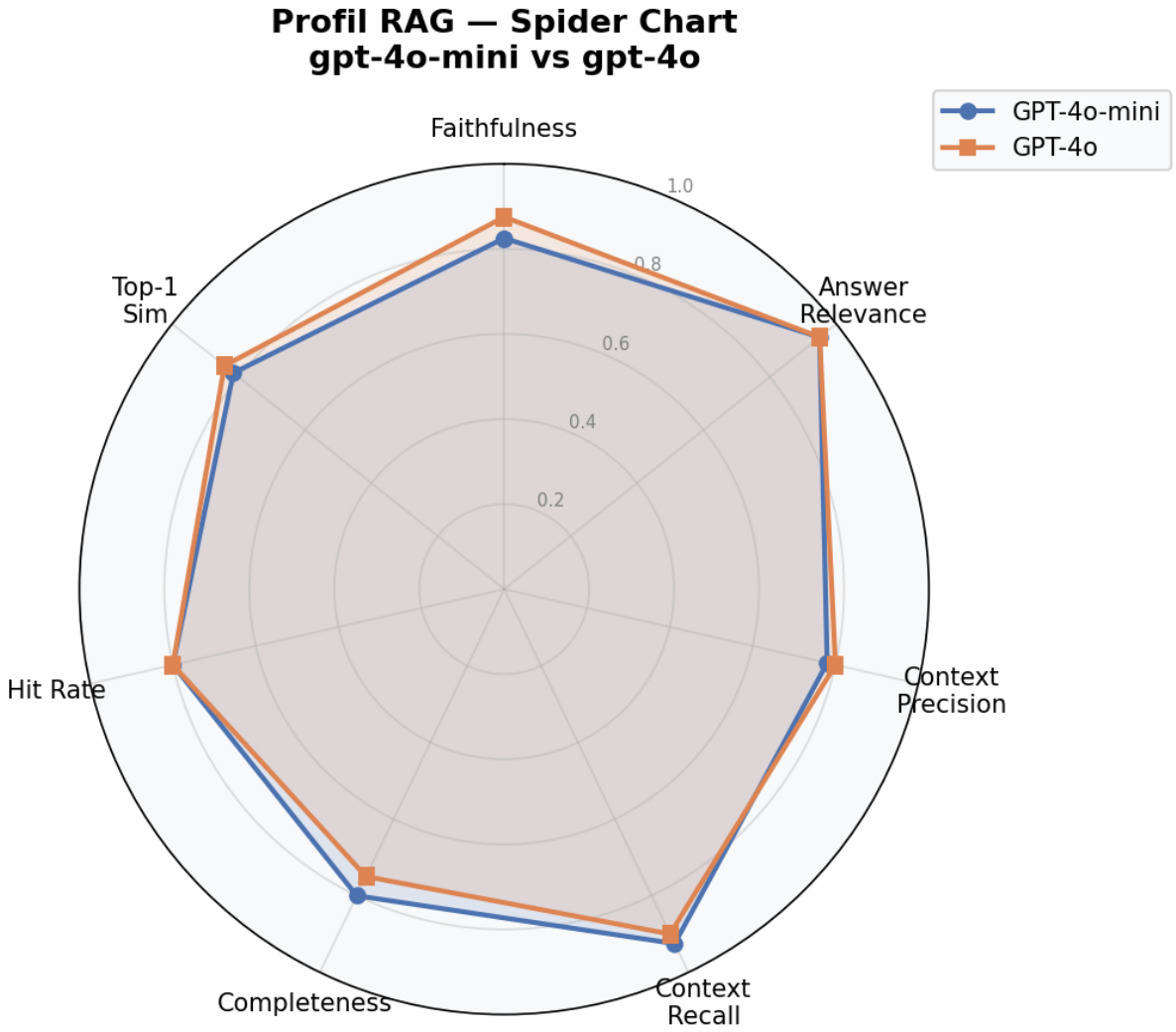
```
v_mini = vals_mini_r + vals_mini_r[:1]
v_gpt4o = vals_gpt4o_r + vals_gpt4o_r[:1]
```

```
fig, ax = plt.subplots(figsize=(7, 7), subplot_kw={"polar": True})
ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
```

```
ax.plot(angles, v_mini, "o-", linewidth=2, color=PALETTE["GPT-4o-mini"],
label="GPT-4o-mini")
ax.fill(angles, v_mini, alpha=0.15, color=PALETTE["GPT-4o-mini"])
ax.plot(angles, v_gpt4o, "s-", linewidth=2, color=PALETTE["GPT-4o"],
label="GPT-4o")
ax.fill(angles, v_gpt4o, alpha=0.15, color=PALETTE["GPT-4o"])
```

```
ax.set_thetagrids(np.degrees(angles[:-1]), radar_labels, fontsize=10)
ax.set_ylim(0, 1)
ax.set_yticks([0.2, 0.4, 0.6, 0.8, 1.0])
ax.set_yticklabels(["0.2", "0.4", "0.6", "0.8", "1.0"], fontsize=7, color="gray")
ax.set_title("Profil RAG — Spider Chart\ngpt-4o-mini vs gpt-4o",
fontweight="bold", pad=20, fontsize=13)
ax.legend(loc="upper right", bbox_to_anchor=(1.3, 1.1))
```

```
plt.tight_layout()
plt.savefig(ROOT / "evaluation" / "fig_radar.png", bbox_inches="tight")
plt.show()
```



Cell 20: ■ Code

```
# ■■ Delta bar chart — amélioration GPT-4o sur mini ████████████████████████
delta_keys = ["faithfulness", "answer_relevance", "context_precision",
"context_recall", "completeness", "hit", "reciprocal_rank",
"mean_similarity", "top1_similarity"]
delta_labels = ["Faithfulness", "Answer Rel.", "Ctx Precision", "Ctx Recall",
"Completeness", "Hit Rate", "MRR", "Mean Sim", "Top-1 Sim"]
deltas = [s_gpt4o[k] - s_mini[k] for k in delta_keys]
```

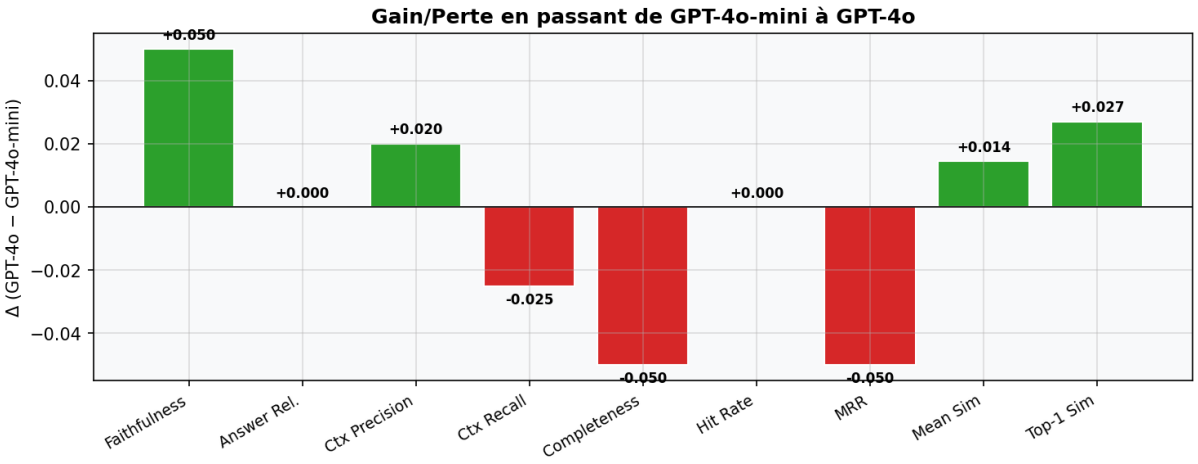
```
fig, ax = plt.subplots(figsize=(10, 4))
colors_d = ["#2ca02c" if d &gt; 0 else "#d62728" for d in deltas]
bars = ax.bar(delta_labels, deltas, color=colors_d, edgecolor="white")
ax.axhline(0, color="black", linewidth=0.8)
ax.set_ylabel("Δ (GPT-4o — GPT-4o-mini)")
ax.set_title("Gain/Perte en passant de GPT-4o-mini à GPT-4o", fontweight="bold")
ax.set_xticklabels(delta_labels, rotation=30, ha="right", fontsize=9)
```

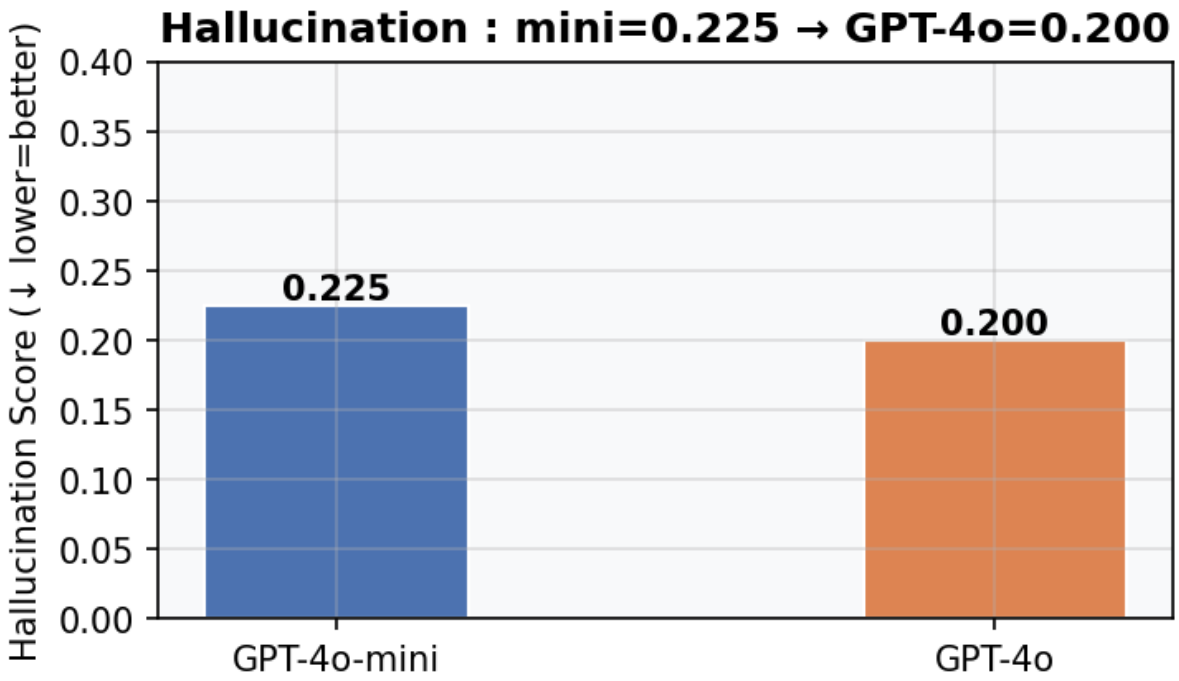
```
for bar, d in zip(bars, deltas):
va = "bottom" if d &gt;= 0 else "top"
offset = 0.002 if d &gt;= 0 else -0.002
ax.text(bar.get_x() + bar.get_width()/2, d + offset, f"{d:+.3f}",
ha="center", va=va, fontsize=8, fontweight="bold")
```

```
plt.tight_layout()
plt.savefig(ROOT / "evaluation" / "fig_delta.png", bbox_inches="tight")
plt.show()
```

```
# ■■ Hallucination comparison
████████████████████████████████████████████████
fig2, ax2 = plt.subplots(figsize=(5, 3))
models = ["GPT-4o-mini", "GPT-4o"]
hallu = [s_mini["hallucination_score"], s_gpt4o["hallucination_score"]]
ax2.bar(models, hallu, color=[PALETTE["GPT-4o-mini"], PALETTE["GPT-4o"]],
edgecolor="white", width=0.4)
ax2.set_ylim(0, 0.4)
ax2.set_ylabel("Hallucination Score (↓ lower=better)")
ax2.set_title("Hallucination : mini=0.225 → GPT-4o=0.200", fontweight="bold")
for i, v in enumerate(hallu):
```

```
ax2.text(i, v + 0.005, f"{v:.3f}", ha="center", fontweight="bold")
plt.tight_layout()
plt.savefig(ROOT / "evaluation" / "fig_hallucination.png", bbox_inches="tight")
plt.show()
```





Cell 21: ■ Markdown

9. Analyse par Question — Deep Dive (GPT-4o-mini)

Cell 22: ■ Code

```
# Charger les résultats individuels depuis le JSON mini
results = rep_mini.get("results", [])
```

```
# Construire le DataFrame par question
rows = []
for r in results:
rows.append({
"ticket_id": r.get("ticket_id", ""),
"hit": r.get("hit", False),
"rank": r.get("rank", 0),
"rag_score": r.get("rag_score", 0),
"faithfulness": r.get("faithfulness", 0),
"answer_relevance": r.get("answer_relevance", 0),
"hallucination_score": r.get("hallucination_score", 0),
"completeness": r.get("completeness", 0),
"question": r.get("question", "")[:80] + "…" if len(r.get("question","")) &gt; 80
else r.get("question",""),
})
```

```
df_per_q = pd.DataFrame(rows)
```

```
# ■■ Bar chart horizontal — RAG Score par question ■■■■■■■■■■■■■■■■■■■■■■■■■■
fig, ax = plt.subplots(figsize=(11, 5))
```

```
bar_colors = []
for _, row in df_per_q.iterrows():
if not row["hit"]: bar_colors.append("#d62728") # MISS — rouge
elif row["rank"] == 1: bar_colors.append("#2ca02c") # rank 1 — vert
else: bar_colors.append("#ff7f0e") # rank 2+ — orange
```

```
bars = ax.barh(df_per_q["ticket_id"], df_per_q["rag_score"], color=bar_colors,
edgecolor="white")
ax.set_xlim(0, 1.1)
ax.set_xlabel("RAG Score")
ax.set_title("RAG Score par Question (GPT-4o-mini)\n■ Hit rank=1 ■ Hit rank≥2 ■
MISS",
fontweight="bold")
ax.axvline(s_mini["rag_score"], color="navy", linestyle="--", linewidth=1.5,
```

```
label=f"Moyenne = {s_mini['rag_score']:.4f}")
ax.legend()
```

```
for bar, row in zip(bars, df_per_q.itertuples()):
rank_lbl = f"rank={row.rank}" if row.hit else "MISS"
ax.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height()/2,
f"{row.rag_score:.3f} ({rank_lbl})",
va="center", fontsize=8)
```

```
plt.tight_layout()
plt.savefig(ROOT / "evaluation" / "fig_per_question.png", bbox_inches="tight")
plt.show()
```

```
display(df_per_q[["ticket_id","hit","rank","rag_score","faithfulness","answer_relevance","hallucination_score","completeness"]]
.style.format({c: "{:.3f}" for c in ["rag_score","faithfulness","answer_relevance","hallucination_score","completeness"]})
.applymap(lambda v: "background-color: #ffdddd" if v == 0.0 and isinstance(v, float) else "", subset=["faithfulness"])
.set_caption("Métriques par question — GPT-4o-mini")
)
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\idirs\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "&lt;string&gt;", line 49
    display(df_per_q[["ticket_id","hit","rank","rag_score","faithfulness","answer_relevance","hal
           ^
SyntaxError: '(' was never closed
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "c:\Users\idirs\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 408
    exec(source, glb)
    ~~~~^^^^^^^^^^^^^
  File "&lt;string&gt;", line 30, in &lt;module&gt;
  File "c:\Users\idirs\Desktop\M2IA\rag_time\.venv\Lib\site-packages\pandas\core\frame.py", line
    indexer = self.columns.get_loc(key)
  File "c:\Users\idirs\Desktop\M2IA\rag_time\.venv\Lib\site-packages\pandas\core\indexes\range.py
    raise KeyError(key)
KeyError: 'ticket_id'
```

Cell 23: ■ Markdown

10. Hallucination & Complétude — Analyse Fine

Cell 24: ■ Code

```
# ■■ Scatter : hallucination_score vs faithfulness ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
```

```
# Left: hallucination vs faithfulness
ax = axes[0]
ax.scatter(df_per_q["hallucination_score"], df_per_q["faithfulness"],
c=df_per_q["hit"].map({True: "#2ca02c", False: "#d62728"}),
s=120, edgecolors="white", zorder=3)
ax.set_xlabel("Hallucination Score (↓)")
ax.set_ylabel("Faithfulness (↑)")
ax.set_title("Hallucination vs Faithfulness", fontweight="bold")
ax.axvline(0.5, color="gray", linestyle=":", linewidth=0.8)
ax.axhline(0.5, color="gray", linestyle=":", linewidth=0.8)
for _, row in df_per_q.iterrows():
ax.annotate(row["ticket_id"], (row["hallucination_score"], row["faithfulness"]),
textcoords="offset points", xytext=(5, 4), fontsize=7, color="dimgray")
# legend
from matplotlib.lines import Line2D
legend_els = [Line2D([0],[0], marker="o", color="w", markerfacecolor="#2ca02c",
markersize=10, label="Hit"),
Line2D([0],[0], marker="o", color="w", markerfacecolor="#d62728",
markersize=10, label="MISS")]
ax.legend(handles=legend_els)
```

```
# Right: completeness vs context_recall
ax2 = axes[1]
ax2.scatter(df_per_q["completeness"], [r.get("context_recall", 0) for r in
results],
c=df_per_q["rag_score"], cmap="RdYlGn", s=120, edgecolors="white", zorder=3,
vmin=0.6, vmax=1.0)
ax2.set_xlabel("Completeness")
```

```
ax2.set_ylabel("Context Recall")
ax2.set_title("Completeness vs Context Recall\n(couleur = RAG Score)",
fontweight="bold")
sm = plt.cm.ScalarMappable(cmap="RdYlGn", norm=plt.Normalize(0.6, 1.0))
sm.set_array([])
plt.colorbar(sm, ax=ax2, label="RAG Score")
```

```
# Régression linéaire
compl = df_per_q["completeness"].values.astype(float)
crec = np.array([r.get("context_recall", 0) for r in results], dtype=float)
if len(compl) &gt; 1:
m, b = np.polyfit(compl, crec, 1)
xs = np.linspace(compl.min(), compl.max(), 50)
ax2.plot(xs, m*xs + b, "--", color="steelblue", linewidth=1.5,
label=f"y={m:.2f}x+{b:.2f}")
ax2.legend(fontsize=8)
```

```
plt.tight_layout()
plt.savefig(ROOT / "evaluation" / "fig_hallucination_completeness.png",
bbox_inches="tight")
plt.show()
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\idirs\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "&lt;string&gt;", line 6, in &lt;module&gt;
  File "c:\Users\idirs\Desktop\M2IA\rag_time\.venv\Lib\site-packages\pandas\core\frame.py", line
    indexer = self.columns.get_loc(key)
  File "c:\Users\idirs\Desktop\M2IA\rag_time\.venv\Lib\site-packages\pandas\core\indexes\range.py
    raise KeyError(key)
KeyError: 'hallucination_score'
```

Cell 25: ■ Code

```
try:
from wordcloud import WordCloud
```

```
# Collecter tous les missing_aspects
all_missing = []
```

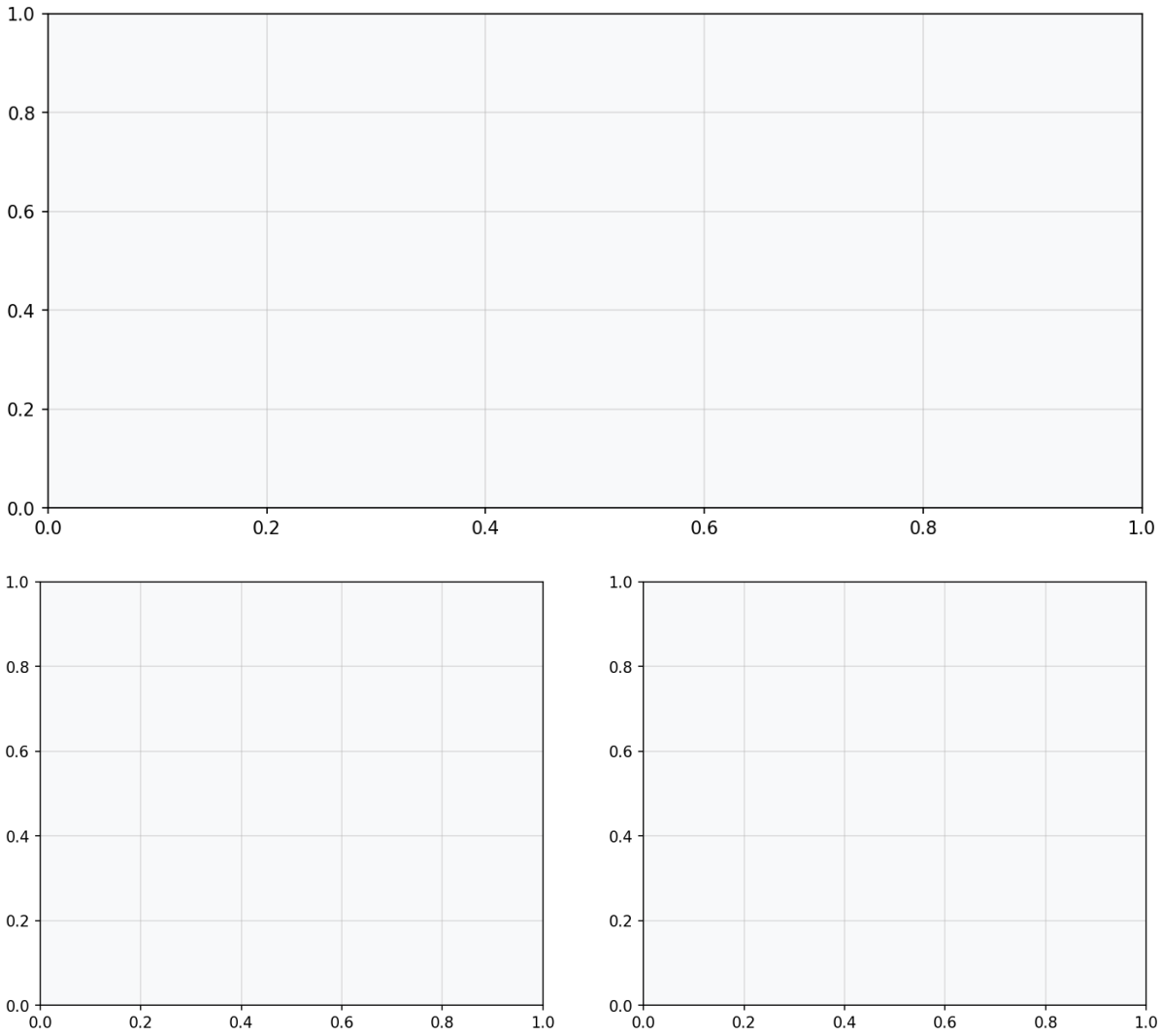
```
for r in results:
ma = r.get("missing_aspects", [])
if isinstance(ma, list):
all_missing.extend([str(x) for x in ma if x])
elif isinstance(ma, str) and ma.strip():
all_missing.append(ma)
```

```
if all_missing:
text = " ".join(all_missing)
wc = WordCloud(width=900, height=400, background_color="white",
colormap="Blues", max_words=80).generate(text)
fig, ax = plt.subplots(figsize=(12, 5))
ax.imshow(wc, interpolation="bilinear")
ax.axis("off")
ax.set_title("Missing Aspects — WordCloud (GPT-4o-mini)", fontweight="bold",
fontsize=14)
plt.tight_layout()
plt.savefig(ROOT / "evaluation" / "fig_missing_aspects_wordcloud.png",
bbox_inches="tight")
plt.show()
else:
print("■■ Aucun missing_aspect trouvé dans le JSON (champ absent ou vide).")
```

```
except ImportError:
print("■■ wordcloud non installé — pip install wordcloud")
```

Output:

```
■■  Aucun missing_aspect trouvé dans le JSON (champ absent ou vide).
```



Cell 26: ■ Markdown

11. Conclusions & Recommandations

Résultats clés

| Critère | GPT-4o-mini | GPT-4o | Verdict |

|---|---|---|---|

| **RAG Score** | 0.8425 | 0.8458 | ■ Excellent sur les deux modèles |

| **Faithfulness** | 0.825 | 0.875 | GPT-4o +6pp — moins d'hallucinations |

| **Hallucination** | 0.225 | 0.200 | GPT-4o légèrement meilleur |

| **Answer Relevance** | 0.950 | 0.950 | Égalité parfaite |

| **Context Recall** | 0.925 | 0.900 | mini légèrement meilleur |

| **Hit Rate @5** | 0.800 | 0.800 | 8/10 questions récupèrent le bon ticket |

Points forts du pipeline

1. **Embedding nomic-ai/nomic-embed-text-v1.5** performant sur textes anglais et allemands

2. **Stratégie 3 chunks** (issue / resolution / context) améliore le recall

3. **Top-1 Similarity > 0.84** (GPT-4o) confirme la qualité de la vectorisation FAISS IndexFlatIP

Axes d'amélioration

- **2 MISS sur 10** → reranking cross-encoder (ex. `cross-encoder/ms-marco-MiniLM-L-6-v2`)

- **Multilinguisme** → dataset > 50% EN ; envisager un modèle multilingue dédié (BGE-M3 avec GPU)

- **Completeness GPT-4o = 0.75** → enrichir les chunks contexte avec subject + tags

- **Evaluation set** → passer à 50–100 questions pour des métriques plus stables

Décision de déploiement

> RAG Score ≥ 0.80 sur les deux modèles → **pipeline validé pour un déploiement en production avec GPT-4o-mini** (rapport coût/performance optimal).